SwaraSetu — Agent-Ready Dataset & Benchmark Plan

0. Mission

Improve the SwaraSetu healthcare triage system by:

1. Auditing the existing repository and current dataset/model.
2. Finding and integrating relevant public medical, triage, multilingual, Indian-language and speech datasets.
3. Creating a clean unified dataset pipeline.
4. Creating a strict leakage-free train/validation/test split.
5. Establishing a reproducible baseline.
6. Training an improved model.
7. Benchmarking BEFORE vs AFTER.
8. Performing external validation on datasets that were NOT used for training.
9. Prioritizing HIGH-risk patient recall over raw accuracy.
10. Producing reproducible benchmark reports and visualizations.

Do NOT fabricate benchmark results.

---

1. FIRST: Audit the Existing Repository

Start by inspecting the entire repository.

Identify:

- Current architecture
- Current models
- Current datasets
- Dataset formats
- Training scripts
- Inference scripts
- Evaluation scripts
- Existing labels
- Existing language support
- Existing triage categories
- Existing preprocessing
- Existing README/documentation
- Existing model checkpoints
- Existing benchmark numbers

Create:

```
reports/repository_audit.md
```

The audit must answer:

- What is the current input?
- What is the current output?
- Is the system text-based, speech-based, or both?
- How are LOW/MEDIUM/HIGH labels currently defined?
- What model is currently being used?
- What dataset is currently being used?
- How many examples exist?
- What languages exist?
- Is the current dataset synthetic or real?
- What is the current class distribution?
- What is the current accuracy/F1/recall?
- What are the current failure cases?

Do not modify the existing implementation during the initial audit.

---

2. Preserve the Current System as BASELINE

Before changing anything, run the existing pipeline.

Save:

```
reports/baseline_metrics.json
reports/baseline_classification_report.csv
reports/baseline_confusion_matrix.png
```

At minimum calculate:

- Accuracy
- Macro F1
- Weighted F1
- Precision
- Recall
- LOW precision/recall/F1
- MEDIUM precision/recall/F1
- HIGH precision/recall/F1

Most importantly:

```
HIGH_RECALL
```

This is a critical safety metric.

Also calculate:

```
HIGH false-negative count
```

because missing a HIGH-risk case is more important than a small increase in overall accuracy.

---

3. Define the SwaraSetu Label Contract

Do NOT blindly convert medical datasets into LOW/MEDIUM/HIGH.

First document:

```
docs/triage_label_policy.md
```

Define exactly what each class means.

Example:

LOW:

- No immediate danger signs
- Routine/basic care pathway

MEDIUM:

- Needs timely clinical assessment/follow-up
- Not currently classified as an immediate emergency

HIGH:

- Danger signs or urgent/emergency referral requirement

The mapping must be clinically defensible.

Use appropriate clinical guidance such as WHO IMCI for child-health scenarios.

Do NOT invent medical rules.

Clearly distinguish:

```
clinical ground truth
```

from:

```
machine-learning prediction
```

---

4. Dataset Sources

Investigate these sources.

Priority A — Triage

MIETIC

MIMIC-IV-Ext Triage Instruction Corpus.

Use for:

- triage classification
- structured clinical features
- emergency severity benchmarking

Investigate:

- licensing
- access requirements
- fields
- labels
- mapping feasibility

MIMIC-IV-ED

Use for:

- external validation
- emergency department triage
- ESI-based evaluation

Do NOT download or bypass restricted access.

If credentialing is required, document the requirement.

Yale Emergency Triage Dataset

Investigate:

- ESI
- demographic features
- triage variables
- admission outcomes

Use primarily as external validation if label mapping is appropriate.

NHAMCS Emergency Department Data

Investigate:

- emergency department triage
- acuity
- demographic variables
- external validation possibilities

Use only if the label mapping is defensible.

---

5. Indian Multilingual Medical Sources

Investigate:

IHQID

Indian Healthcare Query Intent Dataset.

Use for:

- Indian-language healthcare intent
- symptom/query understanding
- multilingual NLU

Languages may include:

- Hindi
- Tamil
- Telugu

- Bengali
- Marathi
- Gujarati
- English

IndicMedDialog

Use for:

- multilingual medical conversations
- multi-turn understanding
- symptom clarification
- conversational robustness

Do not directly treat dialogue answers as clinical ground truth unless labels are explicitly available.

Health_v2

Use for:

- Hindi → Indian language medical translation
- multilingual health terminology
- public-health language

Useful for augmentation and multilingual normalization.

HiMed

Use for:

- Hindi medical understanding
- Hindi medical QA
- medical-language evaluation

Do not convert QA answers into triage labels without a defensible mapping.

Eka-IndicMTEB

Use for:

- medical terminology normalization
- multilingual symptom understanding

- spelling variations
- colloquial queries
- code-mixed queries

This is especially valuable for ASHA-style inputs.

---

6. Speech Sources

If the current SwaraSetu architecture contains speech recognition:

Investigate:

IndicVoices

Use for:

- Indic ASR robustness
- accents
- spontaneous speech
- speaker diversity

Kathbath

Use for:

- Indic speech recognition
- Tamil/Hindi and other language robustness

Doctor-Patient Indic Speech Dataset

Use for:

- healthcare-specific speech
- doctor-patient conversations
- native-language medical speech

Do NOT claim medical triage accuracy from an ASR dataset.

Use these for:

```
Speech → Text
```

evaluation.

---

7. WHO Clinical References

Investigate:

WHO IMCI

Use as a clinical reference for child-health triage.

Extract relevant:

- danger signs
- cough/breathing classification
- diarrhoea classification
- fever classification
- dehydration indicators
- referral indicators

Do NOT scrape and redistribute copyrighted material unnecessarily.

Create an internal rule representation containing only what is necessary for the implementation.

Example:

```
danger_sign_present = true
→ urgent_referral = true
```

The implementation must clearly state that this is a prototype and not a substitute for professional clinical judgment.

---

8. Dataset Registry

Create:

```
data/dataset_registry.yaml
```

Every dataset must have:

```
name:
source:
url:
license:
```

```
access_requirements:
language:
domain:
size:
label_type:
triage_relevance:
training_allowed:
validation_allowed:
external_test_allowed:
notes:
```

Example:

```
- name: MIETIC
  domain: emergency_triage
  language: english
  label_type: ESI
  triage_relevance: high
  training_allowed: true
  external_test_allowed: true
```

Never download a dataset without recording its license/access conditions.

---

9. Dataset Separation

Create THREE categories.

TRAINING

Datasets allowed for model training.

Example:

```
MIETIC
IHQID
Health_v2
IndicMedDialog
Eka-IndicMTEB
```

Only use examples appropriate to the specific training task.

INTERNAL TEST

Create a completely held-out test set from the SwaraSetu dataset.

Example:

```
70% train
15% validation
15% test
```

Use patient-level splitting wherever patient IDs exist.

EXTERNAL TEST

Never train on these datasets.

Use:

```
MIMIC-IV-ED
Yale ED
NHAMCS
```

where licensing and label compatibility permit.

The purpose is to test generalization.

---

10. Prevent Data Leakage

This is mandatory.

Implement:

```
data/leakage_check.py
```

Check:

- duplicate text
- near-duplicate text
- duplicate patient IDs

- overlapping patient records
- translated duplicates
- train/test overlap
- same case appearing in multiple datasets
- augmentation leakage

Use:

- exact hash matching
- normalized text matching
- fuzzy similarity where appropriate

Generate:

```
reports/leakage_report.json
```

If leakage is discovered:

STOP the benchmark.

Fix the split before reporting results.

---

11. Normalize the Data

Create:

```
src/data/normalize.py
```

Convert datasets into a common schema.

Suggested schema:

```
case_id
source_dataset
language
age
sex
chief_complaint
symptoms
duration
vital_signs
medical_history
```

```
raw_text
normalized_text
triage_label
clinical_label
source_label
split
```

Do not discard original labels.

Keep:

```
source_label
```

and:

```
mapped_label
```

separately.

This makes auditing possible.

---

12. Multilingual Normalization

Create:

```
src/language/normalize.py
```

Support:

- English
- Tamil
- Hindi
- Telugu
- Kannada
- Malayalam
- Bengali
- Marathi
- Gujarati
- Punjabi
- Urdu

where data availability permits.

Also test:

Code-mixed input

Examples:

```
"fever இருக்கு"
"breathing problem இருக்கு"
"காய்ச்சல் with cough"
"மூச்சு விட கஷ்டமா இருக்கு"
```

Do not assume all Indian-language input is grammatically perfect.

Include:

- spelling errors
- transliteration
- colloquial terms
- abbreviations
- code mixing

where legally and ethically appropriate.

---

13. Build a Baseline

Use the existing model first.

Then establish at least one simple baseline.

Examples:

```
Majority classifier
Logistic Regression
Random Forest
Existing model
```

This proves that the AI model is actually improving over simple approaches.

Generate:

```
reports/baseline_comparison.csv
```

---

14. Improved Training Dataset

Create multiple training configurations.

Experiment A

Current SwaraSetu dataset only.

Experiment B

Current dataset + triage data.

Experiment C

Current dataset + triage + multilingual data.

Experiment D

Current dataset + triage + multilingual + terminology augmentation.

Do NOT automatically assume D is best.

Measure every experiment.

---

15. Controlled Experiments

Use fixed:

- random seed
- test set
- evaluation code
- preprocessing
- metrics

Example:

```
SEED=42
```

Keep configuration files:

```
configs/baseline.yaml
configs/experiment_a.yaml
configs/experiment_b.yaml
configs/experiment_c.yaml
configs/experiment_d.yaml
```

---

16. Metrics

Every experiment must report:

Classification

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1
- Weighted F1

Per class

LOW:

- Precision
- Recall
- F1

MEDIUM:

- Precision
- Recall
- F1

HIGH:

- Precision
- Recall
- F1

Safety

- HIGH recall
- HIGH false negatives
- HIGH → LOW errors
- HIGH → MEDIUM errors

The most important dangerous error is:

```
ACTUAL HIGH
PREDICTED LOW
```

Report its count and percentage.

---

17. Multilingual Benchmark

Create separate evaluation subsets:

```
English
Tamil
Hindi
Telugu
Kannada
Malayalam
Bengali
Marathi
Code-mixed
Transliterated
```

For each calculate:

```
Accuracy
Macro F1
HIGH Recall
```

Generate:

```
reports/language_benchmark.csv
```

---

18. Speech Benchmark

If speech is supported:

Pipeline:

```
Audio
 ↓
ASR
 ↓
Transcript
 ↓
Medical normalization
 ↓
Triage model
 ↓
LOW/MEDIUM/HIGH
```

Measure separately:

ASR

- WER
- CER

Triage

- Accuracy
- Macro F1
- HIGH Recall

Do NOT combine ASR errors and triage errors into one unexplained accuracy number.

---

19. Robustness Testing

Create adversarial/realistic test categories:

```
Clean English
Tamil
Hindi
Code-mixed
Transliterated
Spelling errors
```

```
Short input
Long input
Incomplete symptoms
Noisy speech
Different accents
Missing information
Conflicting symptoms
```

Measure performance for each.

---

20. Safety Test Set

Create a small manually reviewed safety set.

Include:

```
Clearly LOW
Clearly MEDIUM
Clearly HIGH
Danger signs
Ambiguous cases
Insufficient information
Conflicting symptoms
```

The model must NOT confidently output LOW when critical information is missing.

Consider an:

```
INSUFFICIENT_INFORMATION
```

internal state even if the final UI eventually asks the ASHA worker for more information.

---

21. Confidence / Escalation

Do not force every case into a class.

Implement:

```
if confidence < threshold:
    request_more_information
```

or:

```
if safety_rule_triggered:
    urgent_referral
```

Clinical safety rules should take priority over model confidence.

---

22. Benchmark Dashboard

Create:

```
reports/dashboard.html
```

Show:

Overall

```
Accuracy
Macro F1
HIGH Recall
```

Before vs After

Bar charts.

Confusion matrix

```
LOW / MEDIUM / HIGH
```

Language performance

```
Tamil
Hindi
English
```

```
Code-mixed
...
```

External validation

```
MIETIC
MIMIC-IV-ED
Yale
NHAMCS
```

where available.

---

23. Required Final Comparison

Create:

```
reports/final_benchmark.csv
```

with:

```
Model
Dataset
Language
Accuracy
Macro_F1
HIGH_Precision
HIGH_Recall
HIGH_F1
HIGH_to_LOW_Error
HIGH_to_MEDIUM_Error
```

Example structure:

```
Baseline,SwaraSetu-Test,English,...
Improved,SwaraSetu-Test,English,...
Improved,External-MIETIC,English,...
Improved,External-MIMIC,English,...
Improved,Language-Test,Tamil,...
Improved,Language-Test,Hindi,...
```

Do not invent missing values.

Use `N/A`.

---

24. Reproducibility

Create:

```
scripts/download_data.sh
scripts/prepare_data.py
scripts/train.py
scripts/evaluate.py
scripts/run_benchmark.py
scripts/check_leakage.py
```

The entire benchmark should be reproducible with a command such as:

```
python scripts/run_benchmark.py --config configs/final.yaml
```

It should produce:

```
reports/
├── baseline_metrics.json
├── final_benchmark.csv
├── classification_report.csv
├── confusion_matrix.png
├── language_benchmark.csv
├── leakage_report.json
├── external_validation.csv
└── dashboard.html
```

---

25. Git Workflow

Do NOT destroy the current working version.

Create:

```
feature/benchmark-dataset-improvement
```

Commit logically:

```
audit: inspect current SwaraSetu pipeline

data: add dataset registry

data: add normalization pipeline

data: add leakage detection

eval: establish baseline benchmark

data: add multilingual benchmark

train: add improved dataset experiment

eval: add external validation

docs: add benchmark methodology
```

Never commit:

- private medical data
- credentials
- API keys
- restricted datasets
- large raw datasets unless explicitly permitted
- downloaded copyrighted data without permission

---

26. Acceptance Criteria

The work is NOT complete until all of the following are true:

[ ] Existing repository audited

[ ] Current baseline reproduced

[ ] Current dataset documented

[ ] Dataset registry created

[ ] Dataset licenses/access requirements documented

[ ] Unified schema created

[ ] Leakage detection implemented

[ ] Train/validation/test split created

[ ] External test set separated

[ ] Multilingual evaluation created

[ ] Tamil benchmark created

[ ] Hindi benchmark created

[ ] Code-mixed benchmark created

[ ] HIGH-risk recall measured

[ ] HIGH → LOW errors measured

[ ] Confusion matrices generated

[ ] Before vs After comparison generated

[ ] External validation performed where possible

[ ] No fabricated metrics

[ ] Reproducible benchmark command works

[ ] README updated

[ ] Limitations documented

[ ] Clinical safety disclaimer documented

---

27. Critical Rules

1. NEVER fabricate medical data.
2. NEVER fabricate benchmark results.
3. NEVER claim clinical accuracy without clinical validation.

4. NEVER leak external test data into training.
5. NEVER blindly merge datasets with incompatible labels.
6. NEVER map ESI → LOW/MEDIUM/HIGH without documenting the mapping.
7. NEVER treat synthetic data as real patient data.
8. NEVER expose patient-identifying information.
9. NEVER bypass dataset access restrictions.
10. ALWAYS report HIGH-risk recall.
11. ALWAYS report dangerous HIGH → LOW errors.
12. ALWAYS keep an untouched external test set.
13. ALWAYS preserve source labels.
14. ALWAYS record dataset provenance.
15. If a dataset is unsuitable, document why and skip it.

---

28. Final Deliverable

At the end, produce:

```
SWARASETU BENCHMARK REPORT

1. Current baseline
2. Dataset sources
3. Dataset sizes
4. Dataset licenses
5. Data preprocessing
6. Leakage checks
7. Training methodology
8. Before vs After metrics
9. Tamil performance
10. Hindi performance
11. Code-mixed performance
12. Speech performance
13. External validation
14. HIGH-risk recall
15. Failure cases
16. Limitations
17. Reproducibility instructions
```

The final conclusion must be based ONLY on measured results.

The primary objective is NOT:

"Get the highest accuracy."

The primary objective is:

"Demonstrate that SwaraSetu is more robust, multilingual, generalizable, and safer at identifying high-risk cases than the baseline."